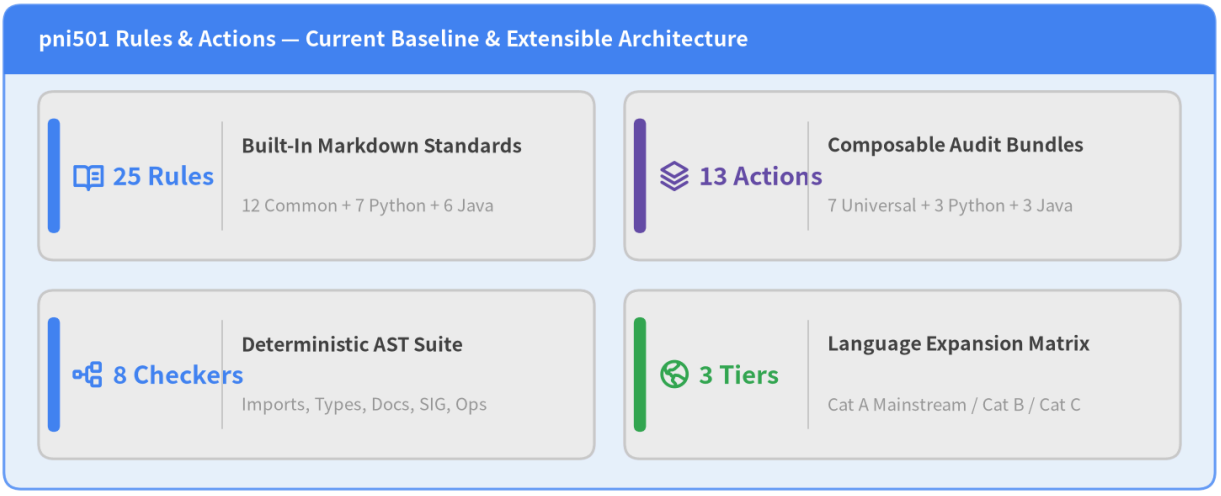
pni501 Rules & Actions — Reference Catalog & Multi-Language Roadmap

Package: `pni501` (`import pni` / CLI: `pni`) | **Theme**: Google Style (`en`) | **Scope**: Built-In Standards, Actions & Expansion Plan

This specification provides the human-oriented engineering reference for **Rules (`_rules`)** and **Actions (`_actions`)** in **Project Nemesis Index (`pni501`)**. It documents both the **currently implemented built-in catalog** (Chapters 1–3) and the **planned multi-language, framework, and external scanner roadmap** (Chapter 4), while AI-executable implementation plans remain cleanly separated under `.agents/plans/`.



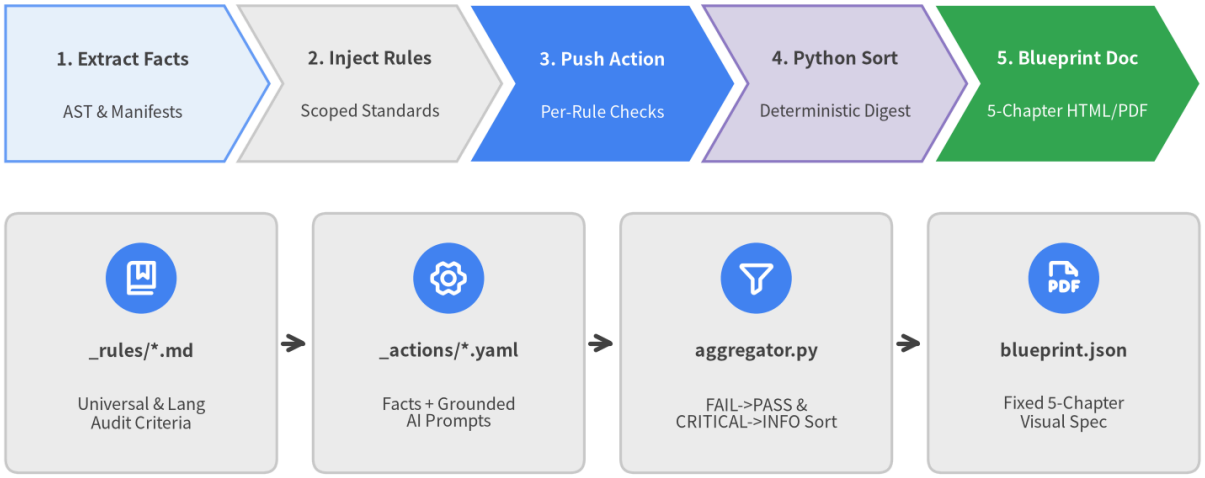
pni501 Rules & Actions Inventory at a Glance: Implemented Core vs. Multi-Language Expansion

Executive Overview: Orthogonal Standards & Deterministic Synthesis

Modern software engineering audits fail when language-specific syntax checks are entangled with architectural governance, or when Large Language Models are asked to both invent

evaluation criteria and summarize findings ad hoc. `pni501` solves both problems through two core design principles:

- **Orthogonal Rule & Action Layering**: Universal engineering standards—such as OpenSSF supply-chain security, 12-Factor configuration, Google SRE production readiness, SIG maintainability thresholds, clig.dev CLI ergonomics, public library hygiene, and Diátaxis documentation—are defined once in `common/*`. Language packs (`python/*`, `java/*`, and future `typescript/*`, `go/*`, `rust/*`) focus strictly on language idioms, build manifests, type systems, and framework rules.
- **Action-Driven Map → Deterministic Python Reduce → 5-Chapter Blueprint**: Every audit action requires structured, schema-validated outputs (`step_score`, per-rule `rule_checks`, and file-grounded `findings`) at push time. Python (`aggregator.py` and `blueprint.py`) deterministically deduplicates, computes module risk hotspots, and sorts all compliance checks (`FAIL → WARN → PASS`) and findings (`CRITICAL → INFO`) before the AI renders the final 5-chapter Drawlib report.



End-to-End Rule-to-Blueprint Pipeline: Grounded Map Step Submissions Followed by Deterministic Python Sorting

Table of Contents

1. Orthogonal Taxonomy & Local Cascade Architecture

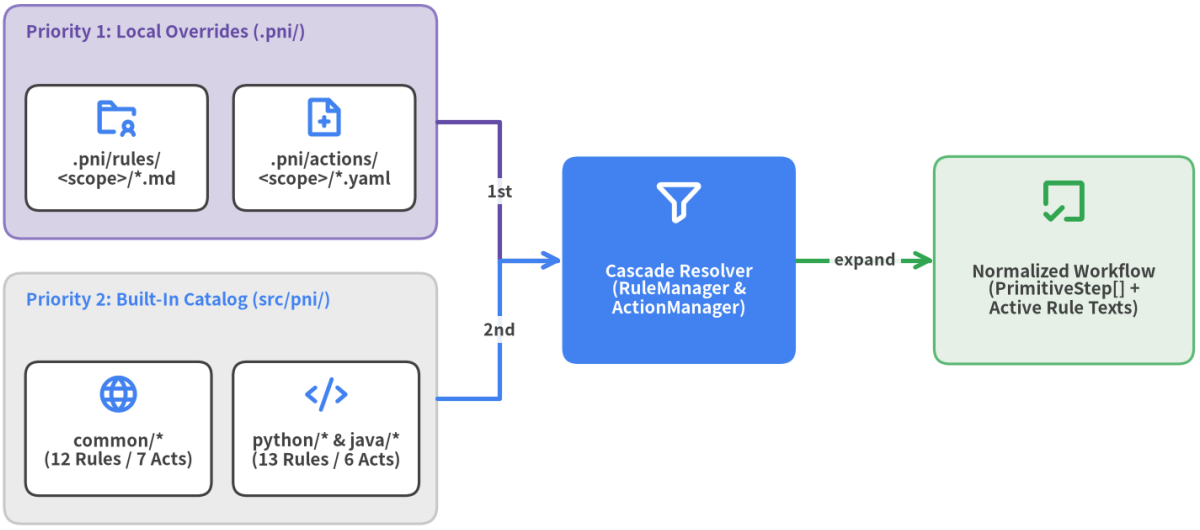
2. Implemented Rules Catalog & Deterministic Fact Grounding

3. Implemented Actions Catalog & Deterministic Map-Reduce Pipeline

4. Future Expansion Roadmap: Multi-Language Matrix, Frameworks & Tooling

1. Orthogonal Taxonomy & Local Cascade Architecture

To scale software health evaluation across diverse runtime archetypes and programming languages without combinatorial duplication, `pni501` organizes both **Rules** (`src/pni/_rules/`) and **Composite Actions** (`src/pni/_actions/`) along two orthogonal axes: **universal engineering governance** versus **language- and framework-specific mechanics**, backed by a deterministic local `.pni/` override cascade.

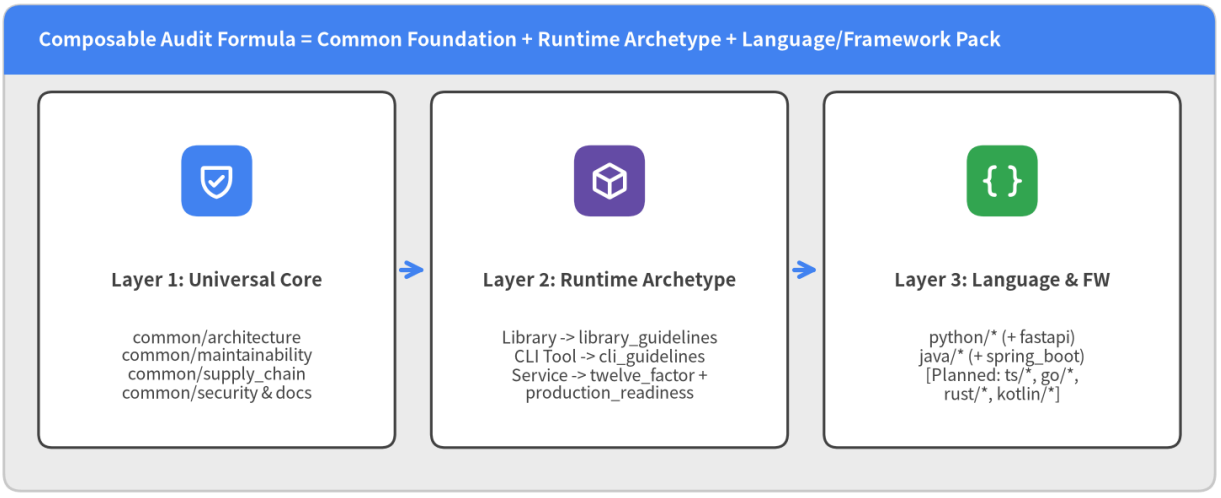


Two-Layer Local Cascade (.pni/ Overrides Over Built-In Packages) Across Common, Python, and Java Scopes

1.1 Orthogonal Separation: Runtime Archetype vs. Language Syntax

A recurring anti-pattern in static analysis toolchains is rewriting architectural, security, and operational rules separately for every programming language. In `pni501`, rules are partitioned strictly by **what kind of engineering invariant they govern**:

1. **Universal Standards (`common/*`)**: Govern invariants that hold true regardless of implementation language—such as layered dependency direction (`common/architecture`), SIG unit complexity limits (`common/maintainability`), OpenSSF lockfile and CI token hygiene (`common/supply_chain`), OWASP secret/injection defenses (`common/security`), 12-Factor statelessness (`common/twelve_factor`), Google SRE I/O timeouts (`common/production_readiness`), `clig.dev` CLI ergonomics (`common/cli_guidelines`), public library API contracts (`common/library_guidelines`), and Diátaxis documentation (`common/documentation`).
2. **Language Core Packs (`<lang>/*`)**: Govern syntax-level idioms, compiler/type-checker strictness, package manager manifests, and unit test conventions—such as PEP 621 `pyproject.toml` + `uv.lock` and PEP 484 annotations in `python/*`, or Maven/Gradle wrappers, Generics safety, and JUnit 5 package mirrors in `java/*`.
3. **Framework Overlay Packs (`<lang>/frameworks/*`)**: Govern framework-specific lifecycle and DI patterns—such as `python/frameworks/fastapi` (lifespan context managers, `Depends()`, Pydantic boundary schemas) and `java/frameworks/spring_boot` (constructor injection over `@Autowired` fields, `@ConfigurationProperties`, transaction boundaries).



Three-Layer Rule & Action Composition: Combining Universal Archetype Governance with Language & Framework Packs

1.2 Scoped Resolution Precedence & CLI Inspection

Both `RuleManager` (`src/pni/_rules/manager.py`) and `ActionManager` (`src/pni/_actions/manager.py`) implement deterministic cascade resolution so teams can customize organizational rules without forking `pni501`:

- **Explicit Scoped IDs (`python/typing`, `common/supply_chain`)**: Resolved by checking `.pni/rules/<scope>/<name>.md` first; if absent, `pni` falls back to the built-in `src/pni/_rules/<scope>/<name>.md`.
- **Unqualified Action IDs in Workflows (`uses: repo_structure_audit`)**: When a workflow declares a top-level `lang: python` (or `lang: java`), unqualified action references automatically search `<lang>/<action_id>.yaml` before falling back to `common/<action_id>.yaml`, checking `.pni/actions/` ahead of built-in `src/pni/_actions/` at each step.
- **Transparent Inspection Commands**: Engineers can inspect which source (`local` vs. `builtin`) wins for every rule and composite action at any time via `pni rules list`, `pni rules show <id>`, `pni actions list`, and `pni actions show <id>`.

2. Implemented Rules Catalog & Deterministic Fact Grounding

`pni501` currently ships **25 built-in Markdown rule files** (`21` evaluation standards and `4` agent/workflow orchestration guides) across `common/` , `python/` , and `java/` . Crucially, every evaluation rule is paired with deterministic static extractors in `ProjectFacts` (`project_inspect`) and `AstFacts` (`ast_analyze`) so AI evaluations are anchored in exact repository evidence rather than unverified LLM impressions.

Rule ID	Anchor Standard	Archetype	Deterministic Grounding Signals (Facts / AST)
common/architecture	Clean / Layered Arch	All Projects	circular_dependencies, internal_import_graph, StructureMap
common/maintainability	SIG / ISO 25010	All Projects	complex_functions (McCabe>7/Cog>15), params>6, global_state
common/supply_chain	OpenSSF Scorecard	All Projects	lockfiles_found, unpinned_deps, ci_supply_chain_issues, LICENSE
common/security	OWASP ASVS Baseline	All Projects	committed_secret_files, hardcoded_secrets, bug_risk_idioms
common/git_workflow	Conventional Repo	All Projects	has_gitignore, committed caches/artifacts, workspace layout
common/documentation	Std Readme & Diataxis	All Projects	readme_headings, readme_line_count, docs_inventory, docstrings
common/twelve_factor	12 / 15-Factor App	Service / API	scattered_env_access, runtime_types, statelessness
common/production_readiness	Google SRE PRR	Service / API	missing_io_timeouts, unstructured_logging, container_hygiene
common/cli_guidelines	clig.dev Guidelines	CLI Tool	entry_points, exit codes, --help/--json, stdout vs stderr
common/library_guidelines	Public SDK Hygiene	Library / SDK	__all__ exports, py.typed, import-time purity, SemVer bounds
common/agent_instruction	PNI Agent Protocol	Meta Guide	Top-level Pull-Push protocol & zero-hallucination rules
common/workflow_guide	PNI Authoring Spec	Meta Guide	Declarative workflow YAML & action composition guide

Implemented Universal Rules Catalog (common/* — 12 Files): Industry Standards & Deterministic Grounding Signals

2.1 Quantitative Thresholds in Universal Standards (`common/*`)

Each universal audit rule translates external engineering specifications into concrete, auditable checkpoints:

- **SIG / ISO 25010 Maintainability (`common/maintainability`)**: Enforces four quantitative code-health invariants—**Unit Size** (`<= 50` LOC target), **Unit Complexity** (`McCabe <= 7`, `cognitive <= 15`), **Unit Interfacing** (`<= 6` parameters; group larger signatures into typed config/DTO objects), and **Module Coupling** (`0` import cycles and `0` mutable module-level globals).
- **OpenSSF Supply-Chain Hygiene (`common/supply_chain`)**: Verifies committed dependency lockfiles (`uv.lock`, `poetry.lock`, `gradle.lockfile`, Maven wrapper), explicit GitHub Actions `permissions:` blocks, SHA-pinned third-party CI actions, non-root pinned `Dockerfile` base images, `SECURITY.md`, `LICENSE`, and Dependabot/Renovate automation.
- **12-Factor & Google SRE Readiness (`common/twelve_factor`, `common/production_readiness`)**: Flags scattered `os.environ` / `System.getenv` reads outside dedicated settings modules, external HTTP/DB/subprocess calls missing explicit `timeout=` budgets, unstructured `print` / `System.out.println` logging in services, and missing health/graceful-shutdown hooks.
- **CLI & Library Archetypes (`common/cli_guidelines`, `common/library_guidelines`)**: Ensures CLIs separate machine-readable `stdout` from diagnostic `stderr` with non-zero exit codes, and ensures libraries expose explicit public facades (`__all__`), ship `py.typed` markers, avoid import-time side effects, and avoid calling `sys.exit()` inside library routines.

2.2 Implemented Language & Framework Rules (`python/*` & `java/*`)

Alongside `common/*`, `pni501` implements full static AST analyzer suites (`LanguageAnalyzerSuite` across **8 abstract checker modules**: `imports`, `typing`, `docs`,

`quality`, `dependencies`, `testing`, `framework`, and `operational`) for **Python** (via Python `ast` + `ast-grep-py`) and **Java** (via `ast-grep-py`).

Rule ID	Scope	Key Engineering Criteria	AST / Manifest Analyzer Signals
python/code_quality	Py Core	Google docstrings, Bugbear idioms	docstring_coverage_pct, bug_risk_idioms
python/typing	Py Core	100% public hints, scoped ignores	type_safety_coverage_pct, weak_types
python/dependencies	Py Core	PEP 621 pyproject + uv.lock hygiene	unpinned_deps, unused/undeclared_deps
python/modern_stack	Py Core	Py 3.11+, pathlib, X|Y, isort order	import_hygiene_pct, unsorted_imports
python/testing	Py Core	pytest mirror layout & parametrize	module_test_coverage_pct, test_smells
python/frameworks/fastapi	Py FW	lifespan, Depends() DI, async purity	framework_issues (on_event, sync IO)
python/workflow_guide	Guide	Python workflow preset & rule guide	pni rules show python/workflow_guide
java/code_quality	Java Core	Javadoc @param sync & Records	docstring_coverage_pct, sig_mismatch
java/typing	Java Core	Generics safety, zero raw/unchecked	type_safety_coverage_pct, weak_types
java/dependencies	Java Core	pom.xml/Gradle + wrapper/lockfiles	build_system, unpinned_dependencies
java/testing	Java Core	src/test/java mirror & JUnit 5 @Test	module_test_coverage_pct, test_smells
java/frameworks/spring_boot	Java FW	Constructor DI & @ConfigProperties	framework_issues (field @Autowired)
java/workflow_guide	Guide	Maven/Gradle/Spring workflow guide	pni rules show java/workflow_guide

Implemented Python (7 Files) & Java (6 Files) Language and Framework Rules Catalog

3. Implemented Actions Catalog & Deterministic Map-Reduce Pipeline

While **Rules** define declarative evaluation criteria, **Actions** (`src/pni/_actions/`) define executable audit units. `pni501` currently provides **8 canonical** `PrimitiveType` **s** and **13 built-in Composite Action YAMLs** (`7` universal `common/` actions, `3` `python/` actions, and `3` `java/` actions), orchestrated through a deterministic Map-Reduce-Blueprint pipeline.

Scope / Action ID	Primitive Sequence	Bound Rules	Validated Push Output
common/architecture_analysis	3x record_architecture	common/architecture	SystemProfile + StructureMap + Flows
common/supply_chain_audit	ai_interpret	supply_chain, security, git_workflow	InterpretationInput (rule_checks[])
common/service_readiness_audit	ai_interpret	twelve_factor, prod_readiness, security	InterpretationInput (12F / PRR)
common/cli_guidelines_audit	ai_interpret	cli_guidelines, documentation	InterpretationInput (CLI UX)
common/library_hygiene_audit	ai_interpret	library_guidelines, docs, maintainability	InterpretationInput (SDK API)
common/emit_score	emit_score	(Consumes CompiledAuditDigest)	ScoreResult (<target>_score.json)
common/write_drawlib_report	write_drawlib_report	(Consumes 5-Chapter Blueprint)	ReportPushInput + HTML/PDF/PNG
python/repo_structure_audit	project_inspect -> ai_interpret	supply_chain, docs, git, sec, py/deps	ProjectFacts + InterpretationInput
python/python_ast_audit	ast_analyze -> ai_interpret	arch, maint, sec, py/{quality,typing,stack}	AstFacts (py) + InterpretationInput
python/python_test_audit	ai_interpret	python/testing	InterpretationInput (pytest audit)
java/repo_structure_audit	project_inspect -> ai_interpret	supply_chain, docs, git, sec, java/deps	ProjectFacts + InterpretationInput
java/java_ast_audit	ast_analyze -> ai_interpret	arch, maint, sec, java/{quality,typing,sb}	AstFacts (java) + InterpretationInput
java/java_test_audit	ai_interpret	java/testing	InterpretationInput (JUnit 5 audit)

Complete Catalog of All 13 Implemented Built-In Composite Actions (common/*, python/*, and java/*)

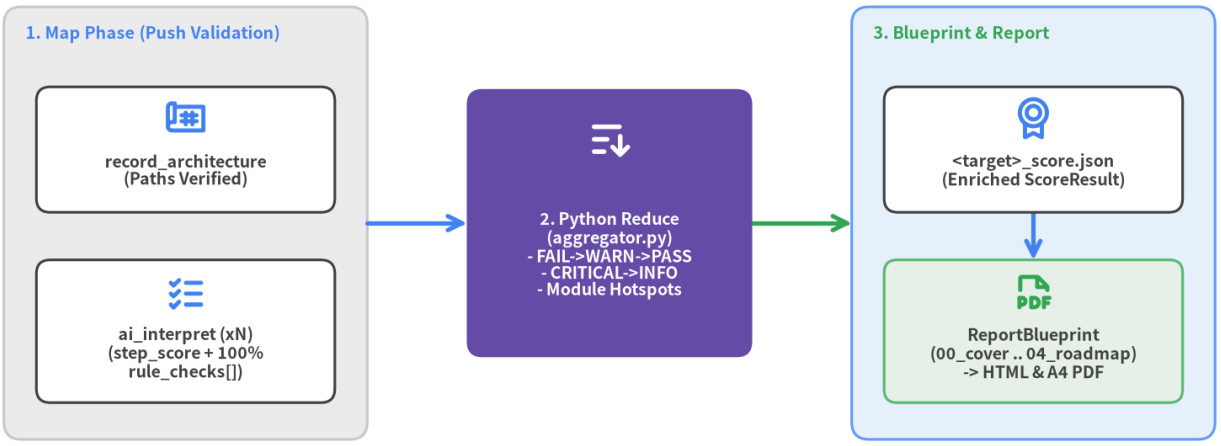
3.1 The 8 Canonical `PrimitiveType`s (`base_model.py`)

Every workflow step—whether declared inline via `action:` or expanded from a composite YAML via `uses:`—executes as one of **8 canonical primitives**:

1. `project_inspect` *(Deterministic Python)*: Scans manifests (`pyproject.toml`, `pom.xml`, `build.gradle`), lockfiles, CI workflows, Dockerfiles, secrets, README headings, and `docs/` trees into `.pni/cache/<target>_facts.json`.
2. `ast_analyze` *(Deterministic Python)*: Runs the 8-dimension `LanguageAnalyzerSuite` (`python` or `java`) to measure exact import hygiene, type coverage, docstring/Javadoc sync, cognitive complexity, coupling cycles, operational timeouts, and test smells.
3. `run_check` *(Deterministic Subprocess)*: Executes whitelisted read-only verification commands and captures exit codes and output summaries into `WorkflowFactsBundle.checks`.
4. `load_rules` *(Context Injection)*: Injects cascaded rule Markdown documents directly into the step context when standalone rule priming is needed.
5. `record_architecture` *(Schema-Grounded AI)*: Records repository-verified `SystemProfile`, `StructureMap` (layers, modules, internal edges), and `ExecutionFlowMap` into `.pni/cache/<target>_architecture.json`, verifying every referenced `path` exists on disk.
6. `ai_interpret` *(Schema-Grounded AI Map Step)*: Evaluates active rules against extracted facts and source files. Enforces submission of a numeric `step_score` (`0–100`), explicit `rule_checks` (`pass` / `warn` / `fail` / `not_applicable`) covering **100% of active step rules**, file-verified `findings`, and concrete `action_items`.
7. `emit_score` *(Deterministic Reduce + Synthesis)*: Presents the pre-sorted `CompiledAuditDigest` to synthesize `overall_score`, `grade`, and executive rationales, then deterministically merges all sorted `audit_dimensions`, `rule_compliance`, `module_hotspots`, `findings`, `action_plan`, and `metrics_snapshot` into `<target>_score.json`.
8. `write_drawlib_report` *(Blueprint-Driven Visual Compiler)*: Generates `.pni/cache/<target>_report_blueprint.json` specifying exact sorted tables, heatmaps, and diagrams for Chapters `00`–`04`, then verifies compiled deliverables (`report.pdf` / `report_html/index.html`) at push time.

3.2 Action-Driven Map-Reduce & 5-Chapter Blueprint Pipeline

Earlier audit pipelines asked the LLM to remember dozens of rules at the end of a run, causing newly added checks to be omitted from prose reports. `pni501` solves this via **deterministic Python aggregation** (`aggregator.py` and `blueprint.py`):



Map-Reduce-Blueprint Loop: Per-Action Structured Push -> Python Deterministic Sort -> 5-Chapter Drawlib Spec

Standard Workflow Archetype Recipes

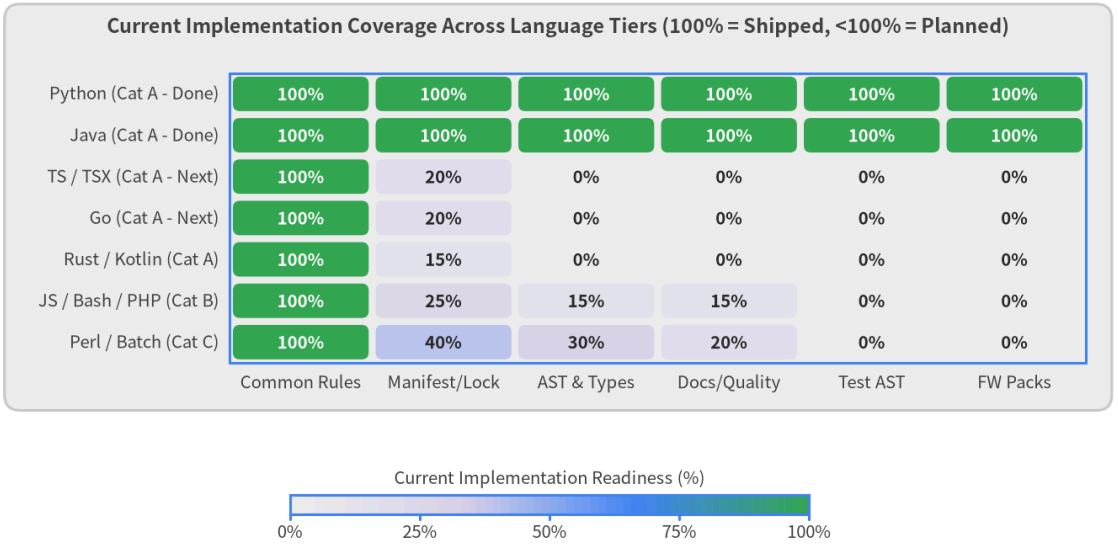
Workflows in `.pni/workflows/*.yaml` compose these 13 actions by project archetype:

- **Library / SDK**: `architecture_analysis` → `repo_structure_audit` → `<lang>_ast_audit` → `library_hygiene_audit` → `<lang>_test_audit` → `emit_score` → `write_drawlib_report`.
- **CLI Application**: `architecture_analysis` → `repo_structure_audit` → `<lang>_ast_audit` → `cli_guidelines_audit` → `<lang>_test_audit` → `emit_score` → `write_drawlib_report`.
- **Service / Web API**: `architecture_analysis` → `repo_structure_audit` → `<lang>_ast_audit` → `service_readiness_audit` → `supply_chain_audit` → `<lang>_test_audit` → `emit_score` → `write_drawlib_report`.

4. Future Expansion Roadmap: Multi-Language Matrix, Frameworks & Tooling

> **Roadmap Status Notice**: Chapters 1–3 document features **implemented and tested today** in `pni501`. Everything in Chapter 4 represents the **planned expansion architecture** for upcoming releases (detailed AI execution specifications are maintained separately under `.agents/plans/`).

Because `common/*` rules (`12` files), universal archetype actions (`7` actions), the SQLite workflow engine, `aggregator.py`, and the 5-chapter `ReportBlueprint` are **100% language-agnostic**, onboarding a new programming language requires zero changes to core orchestration.



Current vs. Planned Multi-Language Capability Matrix Across PNI Audit Dimensions (% Implemented Today)

4.1 Plug-and-Play Language Extension Contract

Adding first-class support for any new language `<lang>` follows a bounded, repeatable four-part module contract without touching `_workflows` or `_reporter`:

1. **Manifest & Lockfile Inspector (** `_actions/project_inspect.py` **)**: Parse the ecosystem's manifest and lockfile (e.g., `package.json` + `pnpm-lock.yaml` / `package-lock.json`, `go.mod` + `go.sum`, `Cargo.toml` + `Cargo.lock`).
2. **8-Checker** `LanguageAnalyzerSuite` **(** `_actions/ast_analyze/<lang>/` **)**: Implement language-specific subclasses of `ast_analyze/base/` (`imports`, `typing`, `docs`, `quality`, `dependencies`, `testing`, `framework`, `operational`) powered by bundled `ast-grep-py` Tree-sitter grammars.
3. **Scoped Rules (** `_rules/<lang>/*.md` **)**: Author `<lang>/code_quality`, `<lang>/typing`, `<lang>/dependencies`, `<lang>/testing`, and optional `<lang>/frameworks/<fw>.md`.
4. **3 Scoped Composite Actions (** `_actions/<lang>/*.yaml` **)**: Add `repo_structure_audit.yaml`, `<lang>_ast_audit.yaml`, and `<lang>_test_audit.yaml`.

4.2 Three-Tier Language Matrix (`Cat A` / `Cat B` / `Cat C`)

Rather than treating legacy scripting languages identically to modern statically typed languages, the roadmap classifies languages into three pragmatic tiers:

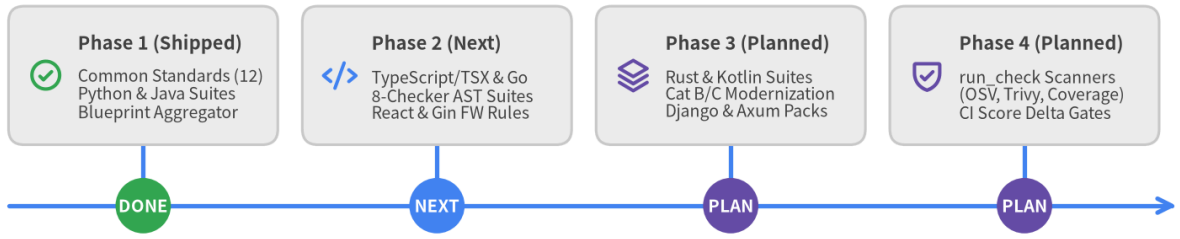
- **Category A — Modern Mainstream Languages (Full 8-Checker** `ast-grep-py` **Suite)**:
  - *Shipped*: **Python** (`stdlib ast` + `ast-grep-py`) and **Java** (`ast-grep-py`).
  - *Phase 2 Priority*: **TypeScript / TSX** (`tsconfig` strictness, `any` / `as unknown` leakage, TSDoc, ESM import cycles, Vitest/Jest AST) and **Go** (`go.mod` / `go.sum`, unchecked `err`, `context.Context` propagation, `godoc`, table-driven `testing.T`).
  - *Phase 3 Priority*: **Rust** (`unsafe` blocks, `unwrap()` / `expect()` in library paths, `Cargo.lock` policy, rustdoc), **Kotlin** (null-safety `!!` abuse, coroutine structured concurrency), **C#**, and **Modern C++ (C++17/20)**.
- **Category B — Transitional & Scripting Languages (AST Audit + Modernization Advisory)**:
  - Covers **JavaScript (** `.js` / `.mjs` **)**, **Bash / Shell (** `.sh` **)**, **PHP (8.2+)**, **Ruby**, and **Raw C**.

    - Combines `ast-grep-py` defect checks (`set -euo pipefail` in Bash, `declare(strict_types=1)` in PHP, Sorbet/RBS hints in Ruby, unsafe buffer calls in C) with explicit **threshold advisories** (e.g., recommending migration from `.js` to TypeScript or rewriting `.sh` scripts exceeding `100` LOC into typed Python/Go CLIs).
- **Category C — Legacy Non-`ast-grep` Languages (Lightweight Scanner + Migration Roadmap)**:
    - Covers **Perl (`.pl` / `.pm` / `.cgi`)** (`use strict; use warnings;` verification) and **Windows Batch / VBScript (`.bat` / `.cmd` / `.vbs`)**.
    - Detected deterministically during `project_inspect` (`legacy_language_findings`) to surface P1/P2 modernization roadmaps without requiring heavy parser dependencies.

4.3 Framework Rule Packs & Sandboxed `run_check` Tooling

Beyond language syntax, two horizontal capability expansions are planned:

- **Framework Rule Packs (`<lang>/frameworks/*`)**: Expanding beyond `python/frameworks/fastapi` and `java/frameworks/spring_boot` to cover **Django** (ORM N+1 & migration safety), **React / Next.js** (hook purity, server/client boundary leaks), **Gin / Echo** (middleware timeout & context handling), and **Axum / Actix-Web**.
- **Deterministic External Scanner Integration (`run_check` Primitive)**: Optional opt-in recipes connecting `run_check` to local security and quality tools—such as `osv-scanner` / `pip-audit` (live CVE lookup against lockfiles), `ruff` / `ty` (compiler-grade diagnostics), `pytest --cov` / `jacoco` (line/branch execution coverage), and `trivy` (container/IaC scanning)—feeding structured counters directly into `WorkflowFactsBundle`.



Phased Evolution Roadmap for pni501 Rules, Actions, Language Suites, and External Scanners